

Heap Data Structures - Detailed Notes

1. Definition and Basics of Heap

A heap is a specialized tree-based data structure that satisfies the heap property:

- In a *Max Heap*, for any given node I , the value of I is greater than or equal to its children.
- In a *Min Heap*, for any given node I , the value of I is less than or equal to its children.

Heaps are commonly implemented as arrays where:

- Left child of node at index i is at $2*i + 1$
- Right child is at $2*i + 2$
- Parent is at $(i - 1) / 2$

2. Heap (Data Structure) vs Heap (Memory Allocation)

In C++, the heap used for dynamic memory (via `new`) refers to a region of memory, not the data structure.

- The *heap data structure* is used for implementing priority queues and scheduling.
- The *heap memory* is where dynamically allocated variables are stored until freed.

3. Applications of Heap

- * Priority Queues: For task scheduling based on priority.
- * Heap Sort: An efficient sorting algorithm ($O(n \log n)$).
- * Graph Algorithms: Dijkstra's and Prim's algorithms use heaps.
- * Median Maintenance: Two heaps can maintain the running median of a stream.
- * Job Scheduling Systems and Bandwidth Management.

4. Heap ADT and Operations

- A. `buildMaxHeap()`: Builds a max heap from an unsorted array.
- B. `maxHeapify(i)`: Fixes heap property at index i by pushing the element down.
- C. `heapifyUp(i)`: Fixes heap property by moving the element at i upwards.
- D. `insert(key)`: Inserts a new key at the end, then restores heap using `heapifyUp`.
- E. `extractMax()`: Removes and returns the maximum element (root), replaces with last node, then `maxHeapify`.
- F. `increaseKey(i, newVal)`: Increases key at i and restores heap using `heapifyUp`.

Heap Data Structures - Detailed Notes

G. `deleteKey(i)`: Deletes element at index `i` by increasing its value to `INT_MAX`, then calling `extractMax`.

H. `heapSort()`: Repeatedly extracts max to sort the array in-place.

I. `findValue(val)`: Linearly searches for a value in the heap.

J. `decreaseKey(i, newVal)`: Decreases key and pushes it down using `maxHeapify`.

5. Edge Cases and Normal Behavior

- Edge Case: Insert on Full Heap: Should return overflow error.
- Edge Case: `extractMax` on Empty Heap: Should return `INT_MIN` or error message.
- Normal Case: Insert/Delete/Heapify: Each maintains the heap invariant using helper functions.
- Special Case: Heap after `HeapSort`: Heap structure is destroyed; to rebuild, call `buildMaxHeap()`.

6. Rebuilding Heap after Heap Sort

After performing `heapSort()`, the array becomes sorted and does not retain heap structure.

To convert it back to a heap:

- Use `buildMaxHeap()` or `buildMinHeap()` depending on the desired heap type.

7. Time Complexities

- Insertion: $O(\log n)$
- Extract Max / Min: $O(\log n)$
- Build Heap: $O(n)$
- Heap Sort: $O(n \log n)$
- Search (unsorted): $O(n)$